

IRMD: Malware variant Detection using opcode Image Recognition

Jixin Zhang, Zheng Qin, Hui Yin, Lu Ou, Yupeng Hu

College of Computer Science and Electronic Engineering, Hunan University, Hunan, China

Hunan Key Laboratory of Big Data Research and Application, Hunan, China

zqin@hnu.edu.cn

Abstract—Malware detection becomes mission critical as its threats spread from personal computers to industrial control systems. Modern malware generally equips with sophisticated anti-detection mechanisms such as code-morphism, which allows the malware to evolve into many variants and bypass traditional code feature based detection systems. In this paper, we propose to disassemble binary executables into opcodes sequences, and then convert the opcodes into images. By using convolutional neural network to compare the opcode images generated from binary targets with the opcode images generated from known malware sample codes, we can detect if the target binary executables is malicious. Theoretical analysis and real-life experiments results show that malware detection using visualized analysis is comparable in terms of accuracy, our approach can significantly improve 15% of detection accuracy when the detection set contains a large quantity of binaries and the training set is much smaller.

Index Terms—Convolutional neural network, Malware detection, Opcode images, Visualized analysis.

I. INTRODUCTION

A. Background

Malware is a major Internet security threat. According to a survey [1], conducted by FireEye in June 2013, 47% of surveyed organizations experienced malware security incidents/network breaches in 2012. McAfee [2] catalogs over 100,000 varied malware samples which have never been identified before, on a daily basis. The number can be interpreted as about 69 new threats every minute or about one threat per second in 2013 [3].

In recent years, most researchers focus on detecting binary malware variants for the binary malware is one of the largest malware family. Researchers have proposed various machine learning approaches like K-Nearest Neighborhood (KNN) [25], Support Vector Machine (SVM) [26], Decision Tree (DT) [27], Random Forest (RF) [28], Naive Bayes (NB) [29] and Clustering [20], [21] to connect unknown samples with known malwares.

Some researchers prefer to use dynamic techniques, which used security tools to collect the methodology calls, operations and raised exceptions of malwares on sandbox or virtual machine platform [4]–[6]. Other researchers prefer to use static analysis [7], [8], [12]–[18] which extracts static malware features to detect/classify malware for improving the accuracy [?]. Nataraj et al. [24] proposed a new method for visualizing and classifying malwares using image processing techniques, which analyze malware binaries as gray-scale images. The

authors found that classification using visualized analysis is comparable in terms of accuracy.

B. Motivations

The operation code (opcode) is widely used for static analysis, related works have demonstrated that the opcode technique can achieve higher accuracy than the other techniques. Santos et al. [16] used n-length opcodes sequences of malware and compared several machine learning algorithms with a real-life malware data set. [17] and [18] also extracted opcode n-grams as features to feed into their malware detectors.

In order to improve the accuracy and reduce the performance overhead, all of above mentioned works choose to extract only a few opcodes and abandon the rest opcodes for search space dimension reduction. In this way, when the training set is much smaller, the detection accuracy might be severely limited. However, as malwares are rapidly growing in volume facing the Internet today, the set of training samples is always smaller than the detection set. On the other side, although their techniques already reduce the search space dimension to improve detection speed, the similarity search can only be performed in a serial manner. It would incur a long delay when the detection set is large. One of our paper's aims is to improve the accuracy and the detection speed.

Malware variants detection problem is similar to image recognition problem, they both need to recognize the variants of original instance, image recognition is based on image similarity while malware variants detection is based on software similarity. Some mature image processing methods are widely used for face recognition, fingerprint identification, etc, these methods perform high accuracy in practice. If there is a method that could convert malware binaries to images, which might preserve the similarities between malware variant images and their original malware images, the image recognition methods might be helpful. To improve the opcode based malware variant detection techniques, the idea of this paper is to apply image recognition methods to the detection techniques.

C. Our approach

Our approach is inspired by Sirko's work on image feature enhancement [30] and Sun's work [33] on face recognition. This paper proposes to disassemble binary executables into opcodes sequences, and then converts the opcodes into images. The structures of the images are standard. For each pixel

point in a image, the pixel-value is a multiplication with the probability of the opcodes sequence and its information gain. Firstly, to make our images easier recognized and classified. Then, we use histogram normalization, erosion and dilation to enhance the contrast between malware images and benign images. Finally, we use convolutional neural network (CNN) to recognize malware images.

D. Feasibility

From the related works [16]–[18], we have known that the probabilities and information gains of opcodes from original malware binaries and their malware variants are relevant. The pixel matrixes of our images as representatives of binaries are reconstructed by the probabilities and information gains. So the original malware images and malware variant images could be relevant. The image processing methods can enhance the contrast between malware images and benign images, which might be useful to classify the images. The CNN is a deep multi-layer NN with a brain inspired architecture motivated by vision tasks, the existing works report that the CNN method has achieved promising results in relevant image recognition recently, it can be used to recognize relevant malware variant images. Through comparing with the state-of-art approaches, the experiments demonstrate that our approach can improve the accuracy and the detection speed.

E. Contributions

The main contributions of the paper are summarized as follows. 1)Our approach transforms malware variant detection problem to image recognition problem, the images which are generated from binaries can be easily recognized and classified. 2)Our approach can significantly improve the accuracy and the detection speed. 3)The experiments demonstrate when the training set is much smaller, the accuracy ratio of our approach is more than 95% while the accuracy ratios of the comparison approaches are less than 80%, and the detection speed of our approach is competitive.

II. PROBLEM STATEMENT

As malwares are rapidly growing in volume facing the Internet today, the set of training samples is always much smaller than the detecting set. When the detection set contains a large quantity of binaries and the training set is much smaller (the $\frac{\text{training}}{\text{training}+\text{detection}}$ ratio is $\frac{1}{24}$), the detection accuracy of the above mentioned opcode based Data Mining Malware Detection (DMMD) techniques might be severely limited, as shown in Table I.

TABLE I
THE ACCURACY OF SEVERAL APPROACHES

approach	accuracy
DMMD(KNN)	79.9%
DMMD(DBC)	78.0%
DMMD(NB)	77.3%

In this work, we try to solve the problem of detecting malware variants with much smaller training sets by image

processing techniques. We convert binaries to standard images as inputs.

Definition 1. Let Y be a training data set which is much smaller, $Y=y_1, y_2$, where y_1 represents the malware data set, y_2 represents the benign data set.

Definition 2. Let x_j be a binary binary, if x_j is a malware binary, x_j belongs to y_1 , if x_j is a benign binary, x_j belongs to y_2 .

Definition 3. Let op_k be the operation codes (opcodes) which are decompiled from binaries.

Definition 4. Let os_i be the operation code (opcode) sequences of length 2, $os_i=\langle op_j, op_k \rangle$.

Definition 5. Let $freq(os_i|x_j)$ be the frequency of os_i in x_j .

Definition 6. Let $p(os_i|x_j)$ be the probability function of os_i in x_j , according to $freq(os_i|x_j)$.

Definition 7. Let $w(os_i)$ be a the information gain function of os_i .

Definition 8. Let $im(x_j)$ be an image matrix generated from x_j , according to $p(os_i|x_j)$ and $w(os_i)$.

Definition 9. Let $im_{enhance}(x_j)$ be an enhanced image by using image feature enhancement.

We ascertain a malware variant is to check if an enhanced malware variant image $im_{enhance}(x_j)$ is sufficiently similar to any previously-seen enhanced malware image in our training data set, a high similarity identifies a malicious variant.

III. OVERVIEW OF METHODOLOGY

Our approach transforms the malware variant detection problem to the image recognition problem, the methodology of our approach solves the problem by using image processing techniques, the methodology includes the following three key steps.

Step 1: Unpacking, decompiling and opcodes sequences.

Some binaries are obfuscated and packed, both making disassembly harder and increasing the variability of code within the same family, but many antivirus techniques and tools implement static unpacking for known packers [9]–[11]. Firstly, we unpack the obfuscated and packed binaries by using both static and dynamic unpacking techniques. Then, we decompile the unpacking binaries to extract their opcodes sequences. We built a opcode profile for each binary, where each profile contains a list of the opcode sequences of length 2 and their frequencies.

Step 2: Binary image reconstruction and enhancement.

Our methodology converts and enhances binaries to standard images as inputs. The images are reconstructed by opcodes sequences of length 2 which are illustrated in step 1. Each element value of an image matrix is a multiplication with the probabilities of the opcodes sequences and their information gains. We use histogram normalization, dilation [23] and erosion [31] to enhance the contrast between malware variant images and benign images.

Step 3: Malware variant image recognition. Once we have reconstructed and enhanced binary images, then we use convolutional neural network (CNN) as a learning approach to recognize malware variant images. Our CNN method has three levels, the inputs of hidden units in layer m are from a subset

```

mov    eax, 0007BB01
mov    ecx, 3E8D0001
call   058A2E97
cmp    al, 24
je     000000A0
mov    ah, 0E

```

Fig. 1. Assembly code example

of units in layer m-1, units that have spatially contiguous receptive fields. For each image as an input, the output of the CNN method is a set of features, the CNN method reduce the dimension of features by mean pooling. Through using softmax classifier, we ascertain a malware variant image is to check if the probability of malware is larger than the probability of benign.

IV. UNPACKING, DECOMPILING AND OPCODES SEQUENCES

Unpacking. Some malware binaries are obfuscated and packed, both making disassembly harder and increasing the variability of code within the same family. We unpack the obfuscated and packed binaries through using both static unpacking techniques PEiD and dynamic unpacking techniques PolyUnpack.

Decompiling. To gather the opcodes sequences of the binaries in the data sets, we decompile the malware binaries and benign binaries by using a decompiler W32asm, we do not use many different decompilers to avoid distortion of the results.

Opcodes sequences. To represent malware binaries using opcodes sequences, we built a opcodes profile for each malware binary, where each profile contains a list of the opcodes sequences of length 2 and their frequencies. We select 2-tuple opcodes sequences, the reason of not extracting further opcode sequence lengths is that the underlying complexity of machine learning methods and the huge amount of features obtained would render the extraction very slow. Assembly code snippet shown in Fig. 1, the above mentioned opcodes sequences can be generated as: $os_1 = \langle mov, mov \rangle$, $os_2 = \langle mov, call \rangle$, $os_3 = \langle call, cmp \rangle$, $os_4 = \langle cmp, je \rangle$, $os_5 = \langle je, mov \rangle$.

V. BINARY IMAGE RECONSTRUCTING AND ENHANCEMENT

In this section, our approach reconstructs and enhances binaries to standard images as our inputs.

A. Binary image reconstructing

The binary image matrixes are reconstructed by these opcodes sequences with their probabilities and information gains. The image matrix is shown in Fig. 2, each opcodes sequences of length 2 can be matched to one of the elements in the

	op1	op2	...	opk	...
op1	$\langle op1, op1 \rangle$	$\langle op1, op2 \rangle$	...	$\langle op1, opk \rangle$	...
op2	$\langle op2, op1 \rangle$	$\langle op2, op2 \rangle$	...	$\langle op2, opk \rangle$	...
...	...	...	...	...	...
opj	$\langle opj, op1 \rangle$	$\langle opj, op2 \rangle$	...	$\langle opj, opk \rangle$	...
...	...	...	...	...	...

Fig. 2. Binary image matrix

matrix according to $os_i = \langle op_j, op_k \rangle$, as shown in Def. 3, Def. 4. The element value $val(os_i|x_j)$ of the image matrix $im(x_j)$ is calculated by the probabilities $p(os_i|x_j)$ and the information gains $w(os_i)$ of os_i in the binary x_j , according to Def. 1, Def. 2, Def. 5, Def. 6, Def. 7, Def. 8, as shown in Eq. (1):

$$val(os_i|x_j) = p(os_i|x_j)w(os_i) \quad (1)$$

The probabilities $p(os_i|x_j)$ and information gains $w(os_i)$ are calculated by the frequencies $freq(os_i|x_j)$ of the opcodes sequences of length 2, as shown in Eq. (2), Eq. (3), where $p(os_i|y_1)$ be the probability of os_i in the training malware binaries, $p(os_i)$ be the probability of os_i in the whole training binaries, and $p(y_1)$ be the probability of training malware binaries.

$$p(os_i|x_j) = \frac{freq(os_i|x_j)}{\sum_{os_i \in x_j} freq(os_i|x_j)} \quad (2)$$

$$w(os_i) = p(os_i|y_1) \log \left(\frac{p(os_i|y_1)}{p(os_i)p(y_1)} \right) \quad (3)$$

B. Binary image enhancement

To enhance the contrast between malware variant images and benign images, we use histogram normalization, dilation and erosion methods to enhance the binary images. Through image enhancement, the contrast of these special opcode images would be enhanced.

Let $val_{enhance}(os_i|x_j)$ be the pixel-value of the enhanced image $im_{enhance}(x_j)$ which is shown in Def. 9 and a constant coefficient α for image enhancement, the histogram normalization method is according to the equation Eq. (4).

$$val_{enhance}(os_i|x_j) = \alpha \frac{val(os_i|x_j)}{\max(val(os_i|x_j))} 255 \quad (4)$$

To make the binary images $im_{enhance}(x_j)$ conspicuous, we dilate twice and then erode twice the images. Erosion followed by dilation can remove noise while retaining useful information. The dilation function and erosion function are from the OpenCV library [32].

After image enhancement, the contrast of malware variant image and benign image is shown in Fig. 3, the left figure as an example shows what a malware binary image looks like,

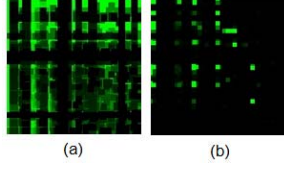


Fig. 3. Malware variant image (a) and benign image (b)

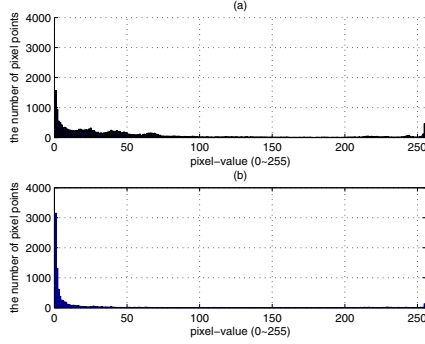


Fig. 4. The examples of histograms of malware variant image (a) and benign image (b)

the right figure as an example shows what a benign binary image looks like. The histograms of the examples are shown in Fig. 4. Our binary images can be easier recognized and classified in visualized analysis.

VI. MALWARE VARIANT IMAGE RECOGNITION

Once we have reconstructed and enhanced binary images, we now discuss how to recognize malware variant images. In this section, we use convolutional neural network (CNN) and softmax classifier for recognizing malware variant images.

The architecture of our CNN model has three levels, both the level 1 and the level 2 are convolution and pooling levels. During the forward pass, each convolutional core is convolved across the width and height of the input volume, computing the dot product between the entries of the filter and the input and producing a 2-dimensional activation map of that convolutional core. Pooling is a form of non-linear down-sampling. The level 3 is full connection level. The softmax classifier classify the malware variant image and benign image.

For each image $img(x_i)$ as an input, the feature of the input is the pixel point of the image matrix. Let $map^{(1)}_j(x_i)$ be the output of level 1, according to Eq. (5), Eq. (6), where the $Val(x_i)$ is the pixel value of $img(x_i)$, the $Pool(x)$ is the mean pooling function and the $Conv(x)$ is the convolution function, the $map^{(1)}_j(x_i)$ will be the input of level 2, and b is a constant value.

$$map^{(1)}_j(x_i) = Pool(Sigmoid(\sum Conv(W^{(1)}Val(x_i))) \quad (5)$$

$$Sigmoid(x) = \frac{1}{1 + e^{-b(x - Avg(x))}} \quad (6)$$

Let $map^{(2)}_j(x_i)$ be the output of level 2, according to Eq. (7), the $map^{(2)}_j(x_i)$ will be the input of level 3. By using convolution and pooling, the dimension of features can be reduced. By using softmax method, the full connection level can map the $map^{(2)}_j(x_i)$ to a target class (malware or benign).

$$map^{(2)}_j(x_i) = Pool(Sigmoid(\sum Conv(W^{(1)}map^{(1)}_j(x_i))) \quad (7)$$

To approach the target, the CNN trains the images and corrects the weight factors of each level through back propagation. For the full connection level, the equation of the softmax classifier is shown in Eq. (8), where u are the original outputs. The output of softmax function is a set of two values, each value represents the probability of the malware image or the benign image.

$$y^{(3)} = \frac{e^{-u}}{\sum e^{-u}} \quad (8)$$

Let W_3 be the weight factors which is connected from the level 3 to the output level, according to Eq. (9), Eq. (10), where the $d^{(3)}_{output}$ is the variance between the level 3 and the output level, the $y^{(3)}$ is the calculated value of the level 3, the y is actual value of the output level and the c is the step length.

$$W^{(3)} = W^{(3)} + c \cdot d_{output} \cdot map^{(2)}_j(x_i) \quad (9)$$

$$d_{output} = (y - y^{(3)}) \cdot y^{(3)} \quad (10)$$

Let $d^{(3)}$ be the variance of level 3, according to Eq. (11). By expanding $d^{(3)}$, we can get $d^{(2)}$ which is the variance of level 2. Let W_2 be the weight factors which is connected from the level 1 to the level 2, according to Eq. (12).

$$d^{(3)} = y^{(3)}(1 - y^{(3)}) \cdot \sum d_{output} \cdot W^{(3)} \quad (11)$$

$$W^{(2)} = W^{(2)} + c \cdot d^{(2)} \cdot Conv(map^{(1)}_j(x_i)) \quad (12)$$

By using inversion convolution and expanding $d^{(2)}$, we can get $d^{(1)}$ which is the variance of level 1. Let W_1 be the weight factors which is connected from the input level to the level 1, according to Eq. (13).

$$W^{(1)} = W^{(1)} + c \cdot d^{(1)} \cdot Conv(img(x_i)) \quad (13)$$

VII. EXPERIMENTS

In this section, we present several real-life experiments to show the efficiency of our approach. In the following, we first present the experiment setup, the data set, the cross validation. Then, we present the approaches for comparison. Finally, we present the accuracy and performance comparison between our approach and the other approaches.

A. The experiments setting

Experiment Setup. We implement our approach and the other approaches on one computer. The version of the CPU is AMD A6-3420M @ 1.50GHz, the RAM is 4.0GB, the operation system is Windows 7. Our approach is developed by C++ programming language and OpenCV 1.0 library.

Data Set. To validate our approach, we use two different data sets to test our malware variant detection methodology: a malware binary data set and a benign binary data set. We downloaded several malware samples from the VxHeavens [22] website to assemble a malware data set of 9,168 about 8GB malicious programs, including 10 large malware families. These malware binaries had already been labelled with their families and variant names by the website. For the benign binary data set, we gathered 8,640 benign binaries from our computers.

TABLE II
THE DATA SET

Malware family	Number
Backdoor	3391
Net-Worm	904
P2P-Worm	360
IRC-Worm	159
IM-Worm	274
Email-Worm	113
Worm	795
Trojan Dropper	908
Trojan-GameThief	742
Trojan Banker	1522
total	9168

Cross Validation. To evaluate the accuracy and performance of the following approaches, we use k-fold cross validation in the experiments. For each group of experiments, the malware data set was split 10 times, and every time it was split into 2 different sets, one for training, the other for detecting.

B. The approaches for comparison

We implement our Malware Variant Detection using Opcode Image Recognition (IRMD) and the opcode sequences for Data Mining based Malware variant Detection technique (DMMD) [16] with real-life malware data set. For IRMD technique, we present the efficiency of our CNN+softmax (IRMD(CNN)) methods. For DMMD technique, we choose their KNN, Naive Bayes (NB) as the classifiers for DMMD. We also implement the density based clustering method for DMMD.

C. Accuracy Comparison

To demonstrate that our approach is optimized through comparing with the state-of-art approaches, we implement our approach IRMD(CNN) and the other approaches DMMD(KNN), DMMD(NB), DMMD(DBC). We have four different data sets, the $\frac{\text{training}}{\text{training+detction}}$ ratios of the data sets are $\frac{1}{24}$, $\frac{1}{12}$, $\frac{1}{6}$, $\frac{1}{3}$.

The accuracy equation is shown in Eq. (14). TPR is the true positive rate, as shown in Eq. (15), where TP is the number of malware cases correctly classified and FN is the number of malware cases misclassified as benign binaries. TNR is the true

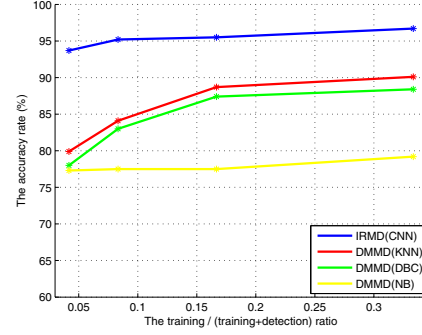


Fig. 5. The accuracy of IRMD and DMMD

negative rate, as shown in Eq. (16), where FP is the number of benign cases incorrectly detected as malware binaries and TN is the number of benign binaries correctly classified.

$$accuracy = \frac{TPR + TNR}{2} \quad (14)$$

$$TPR = \frac{TP}{TP + FN} \quad (15)$$

$$TNR = \frac{1 - FP}{FP + TN} \quad (16)$$

The accuracy comparison results are shown in Fig. 5. From the results, we could easily find out that our approach can significantly improve the accuracy, especially when the training data set is much smaller. By comparing with DMMD(KNN), DMMD(NB) and DMMD(DBC) approaches, our approach IRMD(CNN) achieves 93.7% to 96.7% accuracy while the DMMD(KNN) achieves 79.9% to 90.1%, the DMMD(NB) achieves 77.3% to 79.2% and DMMD(DBC) achieves 78.0% to 88.4%.

The true positive rate (TPR) and the true negative rate (TNR) comparison results are shown in Fig. 6. From the results, we could find out that our approach can significantly improve both TPR and TNR . By comparing with several DMMD approaches, our approach IRMD(CNN) achieves 90.2% to 94.3% TPR and 97.2% to 99.1% TNR while the DMMD(KNN) achieves 75.5% to 88.7% TPR and 84.4% to 91.5% TNR , the DMMD(NB) achieves 77.4% to 81.8% TPR and 75.2% to 77.2% TNR , and the DMMD(DBC) achieves 73.7% to 84.1% TPR and 82.3% to 92.7% TNR .

D. Time Cost Comparison

In this subsection, we present the detecting time cost of our approach IRMD(CNN) and the DMMD approaches, the detecting time cost comparison results are shown in Fig. 7. By comparing with the DMMD(KNN) and DMMD(DBC) approaches, our approach achieves significant speedup for detection.

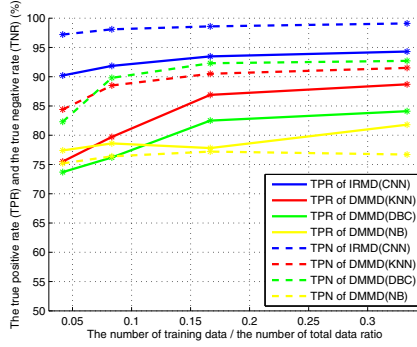


Fig. 6. The true positive rate and the true negative rate of IRMD and DMMD

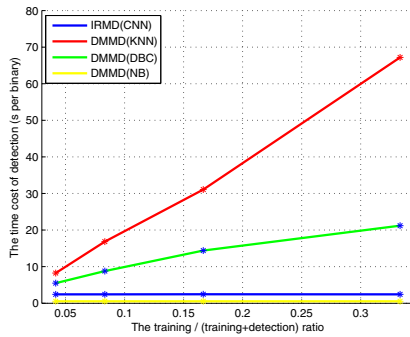


Fig. 7. The time cost of IRMD and DMMD

VIII. CONCLUSIONS

In this paper, we disassemble binary executables into opcodes sequences, and then convert the opcodes into images. We use CNN method to recognize if a binary executable is malicious. The experiments demonstrate that our approach is more efficient than the state-of-art approaches in both accuracy and performance. In the future, as packet malware variant detection is a huge challenge, we will use our image recognition approach to improve it.

ACKNOWLEDGEMENT

This work is partially supported by the National Science Foundation of China under Grant No. 61472131, 61272546, 61300218.

REFERENCES

- [1] Inforisktoday, "2013 Incident Response Survey," <http://www.inforisktoday.in/surveys/2013-incident-response-survey-s-18>, 2013.
- [2] McAfee, "state-of-malware-2013," <http://www.mcafee.com/in/security-awareness/articles/state-of-malware-2013.aspx>, 2013.
- [3] E. Gandotra et al., "Malware 2014 Survey- Analysis and Classification," *Journal of Information Security*, vol. 5, no. 2, pp. 56-64, 2014.
- [4] R. Konrad et al., "Automatic Analysis of Malware Behavior Using Machine Learning," *Journal of Computer Security*, vol. 19, no. 4, pp. 639-668, 2011.
- [5] B. Anderson et al., "Graph Based Malware Detection Using Dynamic Analysis," *Journal in Computer Virology*, vol. 7, no. 4, pp. 247-258, 2011.

- [6] I. Gurrutxaga et al., "Evaluation of Malware clustering based on its dynamic behaviour," In *Proc. of Australasian Data Mining Conference*, pp. 163-170, 2008.
- [7] S. Cesare et al., "Control Flow-Based Malware Variant Detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 11, no. 4, pp. 307-317, 2014.
- [8] D. Kong et al., "Discriminant Malware Distance Learning on Structural Information for Automated Malware Classification," *ACM Knowledge Discovery and Data Mining*, vol. 41, no. 1, pp. 1357-1365, 2013.
- [9] R. Lyda et al., "Using Entropy Analysis to Find Encrypted and Packed Malware," *Security & Privacy*, vol. 5, no. 2, pp. 40-45, 2007.
- [10] S. Cesare et al., "Malwise - An Effective and Efficient Classification methodology for Packed and Polymorphic Malware," *IEEE Transactions on Computers*, vol. 62, no. 6, pp. 1193-1206, 2013.
- [11] Hex-Rays, "IDA: About," <https://www.hex-rays.com/products/ida/>, 2015.
- [12] J. Z. Kolter et al., "Learning to detect and classify malicious executables in the wild," *Journal of Machine Learning Research*, vol. 7, no. 4, pp. 2721C2744, 2006.
- [13] R. Perdisci et al., "Boosting scalability in malware collection and analysis using statistical classification of executables," In *Proc. of ACSAC*, pp. 301C310, 2008.
- [14] B. Anderson et al., "Improving malware classification: bridging the static/dynamic gap," In *Proc. of The 5th ACM workshop on Security and artificial intelligence*, pp. 3C14, 2012.
- [15] K. Raman et al., "Selecting features to classify malware," *InfoSec Southwest*, 2012.
- [16] I. Santos et al., "Opcode Sequences as Representation of Executables for Data mining based malware variant Detection," *Information Sciences*, vol. 231, no. 9, pp. 64-82, 2011.
- [17] M. Zolotukhin et al., "Detection of zero-day malware based on the analysis of opcode sequences," In *Proc. of The 11th Annual IEEE CCNC - Security, Privacy and Content Protection*, pp. 386-391, 2014.
- [18] I. Santos et al., "OPEM: A Static-Dynamic Approach for Machine Learning Based Malware Detection," In *Proc. of International Conference CISIS'12*, pp. 271-280, 2013.
- [19] M. Ester et al., "Density-Based spatial clustering of applications with noise (DBSCAN)," In *Proc. of the Second International Conference on Knowledge Discovery and Data Mining*, pp. 226-231, 1996.
- [20] H. Peng et al., "Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226-1238, 2005.
- [21] F. Murtagh et al., "Methods of Hierarchical Clustering," 2011.
- [22] VX Heaven, "Computer virus collection," <http://vxheaven.org/vl.php>, 2015.
- [23] Wikipedia, "Dilation," [https://en.wikipedia.org/wiki/Dilation_\(morphology\)](https://en.wikipedia.org/wiki/Dilation_(morphology)), 2016.
- [24] L. Nataraj et al., "A Comparative Assessment of Malware Classification using Binary Texture Analysis and Dynamic Analysis," In *Proc. of AISC'11*, pp. 21-30, 2011.
- [25] E. Fix et al., "Discriminatory analysis: Nonparametric discrimination," *Small sample performance*, Technical Report Project 21-49-004, Report Number 11, 1952.
- [26] S. Amari et al., "Improving support vector machine classifiers by modifying kernel functions," In *Proc. of Neural Networks 12*, pp. 783-789, 1999.
- [27] E. R. Mantaras et al., "A distance-based attribute selection measure for decision tree induction," *Machine learning*, vol. 12, no. 6, pp. 81-92, 1991.
- [28] A. Cutler et al., "Random forests," *Machine learning*, vol. 45, no. 1, pp. 157-176, 2011.
- [29] N. Friedman et al., "Bayesian network classifiers," *Machine Learning*, vol. 29, no. 2-3, pp. 598-605, 2011.
- [30] S. Molau et al., "Histogram based normalization in the acoustic feature space," *Automatic Speech Recognition and Understanding*, pp. 21-24, 2003.
- [31] Wikipedia, "Erosion," [https://en.wikipedia.org/wiki/Erosion_\(morphology\)](https://en.wikipedia.org/wiki/Erosion_(morphology)), 2016.
- [32] OpenCV, "Eroding and Dilating," http://docs.opencv.org/2.4/doc/tutorials/imgproc/erosion_dilation/erosion_dilation.html, 2016.
- [33] Yi Sun, Xiaogang Wang, Xiaou Tang, "Deep Learning Face Representation from Predicting 10,000 Classes," In *Proc. of the 2014 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1891-1898, 2014.